

SQL PROJECT ON “PIZZA HUT”

Pizza Sales Analysis using PostgreSQL

Developed a SQL-based data analysis project to analyze pizza sales performance using relational database concepts. Imported CSV datasets into PostgreSQL and performed data cleaning, joins, aggregations, and business analysis using SQL queries. The project includes analysis of revenue, top-selling pizzas, customer ordering patterns, category-wise performance, and sales trends using four tables: pizzas, pizza_types, orders, and order_details.

Technologies & Tools Used

- **Database:** PostgreSQL
- **Language:** SQL
- **Data Source:** CSV Files

Four Tables created such as –

Order_details, Pizzas, Pizza_types, Orders

To visualize the data from tables these queries are used -

“SELECT * FROM order_details;”

“SELECT * FROM Pizzas;”

“SELECT * FROM Pizza_types;”

“SELECT * FROM Orders;”

SELECT * FROM order_details;

	order_details_id [PK] integer	order_id integer	pizza_id character varying (30)	quantity integer
1	1	1	hawaiian_m	1
2	2	2	classic_dlx_m	1
3	3	2	five_cheese_l	1
4	4	2	ital_supr_l	1
5	5	2	mexicana_m	1
6	6	2	thai_ckn_l	1
7	7	3	ital_supr_m	1
8	8	3	prsc_argla_l	1
9	9	4	ital_supr_m	1
10	10	5	ital_supr_m	1

SELECT * FROM Pizzas;

	pizza_id character varying (30)	pizza_type_id character varying (30)	size character (10)	price numeric (5,2)
1	bbq_ckn_s	bbq_ckn	S	12.75
2	bbq_ckn_m	bbq_ckn	M	16.75
3	bbq_ckn_l	bbq_ckn	L	20.75
4	cali_ckn_s	cali_ckn	S	12.75
5	cali_ckn_m	cali_ckn	M	16.75
6	cali_ckn_l	cali_ckn	L	20.75
7	ckn_alfredo_s	ckn_alfredo	S	12.75
8	ckn_alfredo_m	ckn_alfredo	M	16.75
9	ckn_alfredo_l	ckn_alfredo	L	20.75
10	ckn_pesto_s	ckn_pesto	S	12.75

SELECT * FROM Pizza_types;

	pizza_type_id character varying (30)	name character varying (100)	category character varying (20)	ingredients character varying (250)
1	bbq_ckn	The Barbecue Chicken Pizza	Chicken	Barbecued Chicken, Red Peppers, Green Peppers, Tomatoes, Red Onions, Barbecue Sauce
2	cali_ckn	The California Chicken Pizza	Chicken	Chicken, Artichoke, Spinach, Garlic, Jalapeno Peppers, Fontina Cheese, Gouda Cheese
3	ckn_alfredo	The Chicken Alfredo Pizza	Chicken	Chicken, Red Onions, Red Peppers, Mushrooms, Asiago Cheese, Alfredo Sauce
4	ckn_pesto	The Chicken Pesto Pizza	Chicken	Chicken, Tomatoes, Red Peppers, Spinach, Garlic, Pesto Sauce
5	southw_ckn	The Southwest Chicken Pizza	Chicken	Chicken, Tomatoes, Red Peppers, Red Onions, Jalapeno Peppers, Corn, Cilantro, Chipotle Sauce
6	thai_ckn	The Thai Chicken Pizza	Chicken	Chicken, Pineapple, Tomatoes, Red Peppers, Thai Sweet Chilli Sauce
7	big_meat	The Big Meat Pizza	Classic	Bacon, Pepperoni, Italian Sausage, Chorizo Sausage
8	classic_dlx	The Classic Deluxe Pizza	Classic	Pepperoni, Mushrooms, Red Onions, Red Peppers, Bacon
9	hawaiian	The Hawaiian Pizza	Classic	Sliced Ham, Pineapple, Mozzarella Cheese
10	ital_cpcllo	The Italian Capocollo Pizza	Classic	Capocollo, Red Peppers, Tomatoes, Goat Cheese, Garlic, Oregano

SELECT * FROM Orders;

	order_id [PK] integer	order_date date	order_time time without time zone
1	1	2015-01-01	11:38:36
2	2	2015-01-01	11:57:40
3	3	2015-01-01	12:12:28
4	4	2015-01-01	12:16:31
5	5	2015-01-01	12:21:30
6	6	2015-01-01	12:29:36
7	7	2015-01-01	12:50:37
8	8	2015-01-01	12:51:37
9	9	2015-01-01	12:52:01
10	10	2015-01-01	13:00:15

Basic Queries :

Q 1. Retrieve the total number of orders placed.

Query -

```
SELECT COUNT(order_id) AS Total_orders  
FROM orders;
```

Output –

	total_orders bigint
1	21350

Q 2. Calculate the total revenue generated from pizza sales.

Query –

```
SELECT ROUND(SUM(order_details.quantity*pizzas.price),2)  
AS Total_Sales  
FROM order_details JOIN pizzas  
ON order_details.pizza_id=pizzas.pizza_id;
```

Output –

	total_revenue numeric
1	817860.05

Q 3. Identify the highest-priced pizza.

Query -

```
SELECT pizza_types.name, pizzas.price
FROM pizza_types JOIN pizzas
ON pizza_types.pizza_type_id=pizzas.pizza_type_id
ORDER BY pizzas.price DESC
LIMIT 1;
```

Output –

	name character varying (100)	price numeric (5,2)
1	The Greek Pizza	35.95

Q 4. Identify the most common pizza size ordered.

Query –

```
SELECT pizzas.size, Count(Order_details.order_details_id)
AS Most_ordered_pizza
FROM pizzas JOIN order_details
ON pizzas.pizza_id=order_details.pizza_id
GROUP BY pizzas.size
ORDER BY Count(order_details.order_details_id) DESC
LIMIT 5;
```

Output –

	size character (10) 🔒	most_ordered_pizza bigint 🔒
1	L	18526
2	M	15385
3	S	14137
4	XL	544
5	XXL	28

Q 5. List the top 5 most ordered pizza types along with their quantities.

Query -

```
SELECT pizza_types.name, SUM(order_details.quantity)
AS Most_ordered_pizza
FROM pizza_types JOIN pizzas
ON pizza_types.pizza_type_id=pizzas.pizza_type_id
JOIN order_details
ON order_details.pizza_id=pizzas.pizza_id
GROUP BY pizza_types.name
ORDER BY SUM(order_details.quantity) DESC
LIMIT 5;
```

Output –

	name character varying (100) 🔒	most_ordered_pizza bigint 🔒
1	The Classic Deluxe Pizza	2453
2	The Barbecue Chicken Piz...	2432
3	The Hawaiian Pizza	2422
4	The Pepperoni Pizza	2418
5	The Thai Chicken Pizza	2371

Intermediate Queries :

Q 1. Join the necessary tables to find the total quantity of each pizza category ordered.

Query -

```
SELECT pizza_types.category, SUM(order_details.quantity)
AS Most_ordered_pizza
FROM pizza_types JOIN pizzas
ON pizza_types.pizza_type_id=pizzas.pizza_type_id
JOIN order_details
ON order_details.pizza_id=pizzas.pizza_id
GROUP BY pizza_types.category
ORDER BY SUM(order_details.quantity) DESC;
```

Output –

	category character varying (20) 🔒	most_ordered_pizza bigint 🔒
1	Classic	14888
2	Supreme	11987
3	Veggie	11649
4	Chicken	11050

Q 2. Determine the distribution of orders by hour of the day

Query -

```
SELECT EXTRACT(Hour FROM order_time) AS Hour, Count(order_id)
AS Order_Count FROM Orders
GROUP BY EXTRACT(Hour FROM order_time);
```

Output –

	hour numeric 🔒	order_count bigint 🔒
1	11	1231
2	23	28
3	18	2399
4	19	2009
5	15	1468
6	9	1
7	21	1198
8	17	2336
9	20	1642
10	13	2455

Q 3. Join relevant tables to find the category-wise distribution of pizzas.

Query -

```
SELECT category, Count(name) FROM pizza_types
GROUP BY category;
```

Output –

	category character varying (20) 🔒	count bigint 🔒
1	Supreme	9
2	Classic	8
3	Veggie	9
4	Chicken	6

Q 4. Group the orders by date and calculate the average number of pizzas ordered per day.

Query -

```
SELECT Round(AVG(quantity),0) AS Avg_pizza_ordered_per_day FROM
(SELECT orders.order_date, SUM(order_details.quantity) AS quantity
FROM orders JOIN order_details
ON orders.order_id = order_details.order_id
GROUP BY orders.order_date) AS order_quantity;
```

Output –

	avg_pizza_ordered_per_day numeric
1	138

Q 5. Determine the top 3 most ordered pizza types based on revenue.

Query -

```
SELECT pizza_types.name, ROUND(SUM(order_details.quantity*pizzas.price),2)
AS Revenue
FROM pizza_types JOIN pizzas
ON pizza_types.pizza_type_id=pizzas.pizza_type_id
JOIN order_details
ON order_details.pizza_id=pizzas.pizza_id
GROUP BY pizza_types.name
ORDER BY ROUND(SUM(order_details.quantity*pizzas.price),2) DESC
LIMIT 3;
```

Output –

	name character varying (100)	revenue numeric
1	The Thai Chicken Pizza	43434.25
2	The Barbecue Chicken Piz...	42768.00
3	The California Chicken Piz...	41409.50

Advanced Queries :

Q 1. Calculate the percentage contribution of each pizza type to total revenue.

Query -

```
SELECT
    pizza_types.category,
    ROUND(
```

```

SUM(order_details.quantity * pizzas.price) * 100 /
(
    SELECT ROUND(SUM(order_details.quantity * pizzas.price), 2)
    FROM order_details
    JOIN pizzas
    ON pizzas.pizza_id = order_details.pizza_id
),
2) AS revenue
FROM pizza_types
JOIN pizzas
    ON pizza_types.pizza_type_id = pizzas.pizza_type_id
JOIN order_details
    ON order_details.pizza_id = pizzas.pizza_id
GROUP BY pizza_types.category
ORDER BY revenue DESC;

```

Output –

	category character varying (20) 🔒	revenue numeric 🔒
1	Classic	26.91
2	Supreme	25.46
3	Chicken	23.96
4	Veggie	23.68

Q 2. Analyse the cumulative revenue generated over time.

Query -

```

SELECT order_date, SUM(revenue) OVER(ORDER BY order_date) AS
cum_revenue

FROM

(SELECT orders.order_date, SUM(order_details.quantity*pizzas.price) AS
revenue

```

```

FROM order_details JOIN pizzas
ON order_details.pizza_id=pizzas.pizza_id
JOIN orders
ON orders.order_id=order_details.order_id
GROUP BY orders.order_date) AS sales;

```

Output –

	order_date date	cum_revenue numeric
1	2015-01-01	2713.85
2	2015-01-02	5445.75
3	2015-01-03	8108.15
4	2015-01-04	9863.60
5	2015-01-05	11929.55
6	2015-01-06	14358.50
7	2015-01-07	16560.70
8	2015-01-08	19399.05
9	2015-01-09	21526.40
10	2015-01-10	23990.35

Q 3. Determine the top 3 most ordered pizza types based on revenue for each pizza category.

Query -

```

SELECT category, name, revenue FROM
(SELECT category, name, revenue,
RANK() OVER(PARTITION BY category ORDER BY revenue DESC) AS rank
FROM
(SELECT pizza_types.category, pizza_types.name,
SUM(order_details.quantity*pizzas.price) AS revenue
FROM pizza_types JOIN pizzas
ON pizza_types.pizza_type_id=pizzas.pizza_type_id
JOIN order_details

```

ON order_details.pizza_id=pizzas.pizza_id

GROUP BY pizza_types.category, pizza_types.name) AS a) AS b

WHERE rank <=3;

Output –

	category character varying (20) 🔒	name character varying (100) 🔒	revenue numeric 🔒
1	Chicken	The Thai Chicken Pizza	43434.25
2	Chicken	The Barbecue Chicken Piz...	42768.00
3	Chicken	The California Chicken Piz...	41409.50
4	Classic	The Classic Deluxe Pizza	38180.50
5	Classic	The Hawaiian Pizza	32273.25
6	Classic	The Pepperoni Pizza	30161.75
7	Supreme	The Spicy Italian Pizza	34831.25
8	Supreme	The Italian Supreme Pizza	33476.75
9	Supreme	The Sicilian Pizza	30940.50
10	Veggie	The Four Cheese Pizza	32265.70
11	Veggie	The Mexicana Pizza	26780.75
12	Veggie	The Five Cheese Pizza	26066.50